

Walk Before You Run!

Concise LLM Reasoning via Reinforcement Learning

Mingyang Song, Mao Zheng
Tencent Hunyuan
nickmysong@tencent.com

Abstract

As test-time scaling becomes a pivotal research frontier in Large Language Models (LLMs) development, contemporary and advanced post-training methodologies increasingly focus on extending the generation length of long Chain-of-Thought (CoT) responses to enhance reasoning capabilities toward DeepSeek R1-like performance. However, recent studies reveal a persistent overthinking phenomenon in state-of-the-art reasoning models, manifesting as excessive redundancy or repetitive thinking patterns in long CoT responses. To address this issue, in this paper, we propose a simple yet effective two-stage reinforcement learning framework for achieving concise reasoning in LLMs, named **ConciseR**. Specifically, the first stage, using more training steps, aims to incentivize the model’s reasoning capabilities via **Group Relative Policy Optimization** with *clip-higher* and *dynamic sampling* components (**GRPO++**), and the second stage, using fewer training steps, explicitly enforces conciseness and improves efficiency via **Length-aware Group Relative Policy Optimization** (**L-GRPO**). **Significantly, ConciseR only optimizes response length once all rollouts of a sample are correct, following the "walk before you run" principle.** Extensive experimental results demonstrate that our **ConciseR** model, which generates more concise CoT reasoning responses, outperforms recent state-of-the-art reasoning models with zero RL paradigm across AIME 2024, MATH-500, AMC 2023, Minerva, and Olympiad benchmarks. The code, training dataset, and model checkpoints will be publicly released¹.

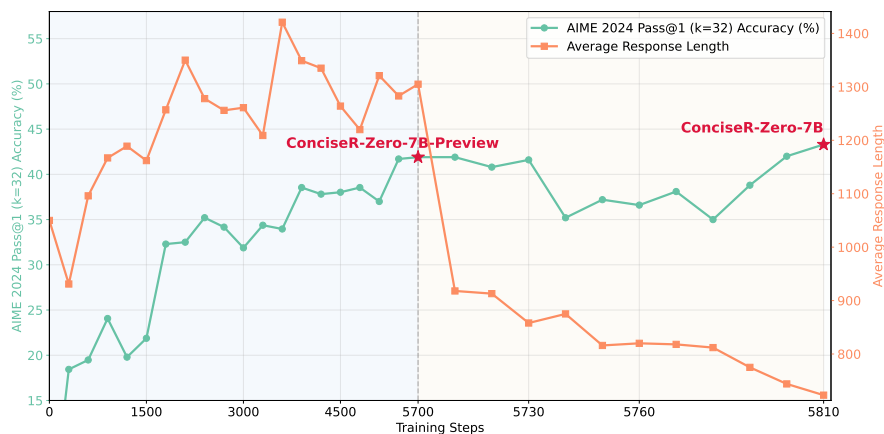


Figure 1: A detailed evaluation of accuracy and response length throughout the training steps.

¹<https://github.com/nick7nlp/ConciseR>

1 Introduction

Test-time scaling [1, 2] has demonstrated a robust correlation between extending the generation length of Chain-of-Thought (CoT) [3] and improving the reasoning capabilities of Large Language Models (LLMs). The advent of large reasoning models, such as GPT-o1 [4] and DeepSeek-R1 [5], represents a significant breakthrough in natural language processing, especially in tackling complex and intricate reasoning tasks. An interesting phenomenon observed during reinforcement learning post-training via Group Relative Policy Optimization (GRPO) [6] is the emergence of an "aha moment" [5], which refers to a pivotal inflection point at which the model spontaneously initiates self-correction behaviors. These emergent behaviors develop autonomously through the model's exploration of the solution space rather than through explicit programming. Prior research [7, 8] has found a distinctive pattern following this moment: the response length of the model tends to increase significantly, accompanied by improvements in overall performance. Despite the lack of a clear understanding of why this occurs, this phenomenon has led many researchers to advocate for longer responses, leveraging additional computational resources in the hope of further enhancing accuracy.

However, generating excessively long CoT reasoning responses substantially increases computational overhead in both the model training and deployment phases. Furthermore, recent studies [9, 10] have discovered an intrinsic overthinking phenomenon in reasoning models, where these models persistently produce verbose rationalizations. This tendency manifests as the inclusion of irrelevant contextual information and unnecessary reflective behaviors. Such information and behaviors not only inefficiently consume computational resources but also compromise reasoning accuracy by causing models to deviate from valid logical pathways to incorrect conclusions.

To address these issues, recent studies [8, 11, 12, 13, 14] are researching efficient reasoning methodologies based on the GRPO algorithm for training the model to produce more concise CoT responses, and have discovered a trade-off between the CoT response length and model reasoning capabilities in most cases, i.e., the shorter the length, the worse the performance. It is understandable that achieving efficient reasoning, improving ability via a more concise CoT, is inherently more challenging. This is in contrast to boosting performance by merely increasing the response length, as the former requires significantly higher model capabilities. Therefore, we highlight the critical importance of the timing for optimizing response length when training using GRPO-based algorithms. Adhering to the "walk before you run" principle, we consider that during training, response length optimization is only enabled when all rollouts for a training sample are correct.

Motivated by this, we propose ConciseR, which is a simple yet effective two-stage reinforcement learning framework for concise reasoning. Specifically, the first stage aims to enhance the model's reasoning capabilities through group relative policy optimization with clip-higher and dynamic sampling techniques. Meanwhile, we introduce the entropy bonus in the first stage, which is used to encourage exploration in policy gradient to incentivize the reasoning capabilities of the model further. The second stage enforces conciseness and improves efficiency via length-aware group relative policy optimization (incorporating max response length to calculate the length reward). Experiments demonstrate that ConciseR is compatible with RL models that incentivize reasoning, achieving reductions in response length while improving accuracy across benchmarks such as AIME 2024, AMC 2023, MATH-500, Minerva, and Olympiad datasets. As shown in Figure 1, ConciseR gradually activates the reasoning ability of the model in the first stage, then rapidly compresses the CoT response length in the second stage to achieve concise reasoning. Notably, the reduction in response length has immediate implications for computational efficiency, resource utilization, and response time, making the approach practically appealing and cost-effective.

2 Preliminary

2.1 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) [15] is one of the policy gradient methods that introduces a clipped surrogate objective for policy optimization. By using clipping to constrain policy updates within a proximal region of the previous policy, PPO stabilizes training and improves sample efficiency.

Specifically, PPO updates the policy by maximizing the following objective:

$$\begin{aligned} \mathcal{J}_{\text{PPO}}(\theta) &= \mathbb{E}_{q \sim \mathcal{D}, o_{\leq t} \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \\ &\min \left[\frac{\pi_{\theta}(o_t | q, o_{\leq t})}{\pi_{\theta_{\text{old}}}(o_t | q, o_{\leq t})} \hat{A}_t, \text{clip} \left(\frac{\pi_{\theta}(o_t | q, o_{\leq t})}{\pi_{\theta_{\text{old}}}(o_t | q, o_{\leq t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_t \right], \end{aligned} \quad (1)$$

where $\pi_{\theta_{\text{old}}}$ is the policy before the update, ε is the clipping hyper-parameter, q indicates the question from the data distribution $\mathcal{D}$, and $\hat{A}_t$ is an estimator of the advantage function of the t -th token. Here, a standard and traditional way to estimate $\hat{A}_t$ is to compute the Generalized Advantage Estimation (GAE) [15] with a learned value model. However, in the context of LLM RL scaling, learning the value model is computationally expensive, so methods that estimate $\hat{A}_t$ without a learned value model are practically preferred.

2.2 Group Relative Policy Optimization (GRPO)

Group Relative Policy Optimization (GRPO) [6] introduces a policy gradient framework that eliminates the reliance on explicit value function by utilizing comparative advantage estimation within a group of responses. This method samples multiple candidate outputs for each input question and computing advantages based on the relative rewards among these candidates within their respective groups. Specifically, GRPO first samples a group of responses $\{o_1, o_2, \dots, o_G\}$ per question and computes their returns $\mathbf{r} = \{r_1, r_2, \dots, r_G\}$, then calculates the advantage of the i -th response o_i as,

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}. \quad (2)$$

Similar to PPO, GRPO uses a clipped objective with KL penalty and optimizes the policy model π_{θ} by maximizing the following objective:

$$\begin{aligned} \mathcal{J}_{\text{GRPO}}(\theta) &= \mathbb{E}_{q \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \\ &\frac{1}{G} \sum_{i=1}^G \left\{ \min \left[\tau_i(\theta) \hat{A}_i, \text{clip}(\tau_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right] - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta} || \pi_{\text{ref}}] \right\}, \end{aligned} \quad (3)$$

where

$$\tau_i = \frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{\text{old}}}(o_i|q)}, \quad \mathbb{D}_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}}) = \frac{\pi_{\text{ref}}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{\text{ref}}(o_i|q)}{\pi_{\theta}(o_i|q)} - 1. \quad (4)$$

Here, π_{ref} represents the reference model and the term $\mathbb{D}_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}})$ indicates a KL penalty term to limit how much the trained model π_{θ} can deviate from the reference model π_{ref} .

2.3 Decouple Clip and Dynamic Sampling Policy Optimization (DAPO)

An in-depth analysis [12] reveals that the naive GRPO baseline suffers from several significant issues, such as entropy collapse, reward noise, and training instability. To address this issue, DAPO introduces four key techniques to make RL shine in the long-CoT RL scenario, including

- *Clip-Higher*, which enhances the diversity of the model and avoids entropy collapse.
- *Dynamic Sampling*, which improves training efficiency and stability.
- *Token-Level Policy Gradient Loss*, which plays a crucial role in reinforcement learning scenarios involving Long-CoT reasoning responses.
- *Overlong Reward Shaping*, a length-aware penalty mechanism designed to shape the reward for truncated samples to reduce reward noise and stabilize training.

Similar to GRPO, DAPO estimates the advantage in a group-relative manner and optimizes the policy model via the following objective,

$$\begin{aligned} \mathcal{J}_{\text{DAPO}}(\theta) &= \mathbb{E}_{q \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \frac{1}{G} \sum_{i=1}^G \left\{ \min \left[\tau_i(\theta) \hat{A}_i, \text{clip}(\tau_i(\theta), 1 - \varepsilon_l, 1 + \varepsilon_h) \hat{A}_i \right] \right\} \\ &\text{s.t. } 0 < |\{o_i | \text{is_equivalent}(o_i, a)\}| < G, \end{aligned} \quad (5)$$

where ε_l and ε_h indicate the lower and higher clipping range.

3 Methodology

Our primary goal is to let LLM generate a more concise CoT response without sacrificing the model’s performance. To this end, we propose a novel two-stage reinforcement learning training paradigm guided by the principle of "*Aim for 100% accuracy first; speed comes with mastery.*" Specifically, the first stage aims to incentivize the reasoning capabilities of the base model via group relative policy optimization with clip-higher and dynamic sampling, thus ensuring accuracy and robust model reasoning. Subsequently, the second stage explicitly enforces conciseness and improved efficiency via length-aware group relative policy optimization, aligning with our objective of achieving mastery through precision, where conciseness naturally follows from reliable and accurate reasoning.

3.1 Group Relative Policy Optimization with Clip-Higher and Dynamic Sampling (GRPO++)

In the first stage, the model is trained to incentivize the reasoning capabilities, which aims to enhance the model’s problem-solving capacity, with an expected increase in response length as GRPO mostly encounters negative rewards and encourages the trained model toward longer responses. To this end, in this paper, we adopt GRPO with two key components of DAPO, clip higher and dynamic sampling, and further introduce an entropy bonus to encourage greater exploration capability in the model, named GRPO++. Similar to the original approach, GRPO++ estimates the advantage in a group-relative manner and optimizes the policy model using the following objective:

$$\begin{aligned} \mathcal{J}_{\text{GRPO++}}(\theta) &= \mathbb{E}_{q \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \\ &\frac{1}{G} \sum_{i=1}^G \left\{ \min \left[\tau_i(\theta) \hat{A}_i, \text{clip}(\tau_i(\theta), 1 - \varepsilon_l, 1 + \varepsilon_h) \hat{A}_i \right] + \alpha \mathbb{H}(\pi_\theta) \right\}, \\ \text{s.t. } &0 < |\{o_i | \text{is_equivalent}(o_i, a)\}| < G, \end{aligned} \quad (6)$$

where $\alpha \mathbb{H}(\pi_\theta)$ denotes the entropy bonus.

Table 1: Analyze the character-level output length and the frequency of reflections of correct and incorrect responses for DEEPSSEEK-R1-DISTILL-QWEN-1.5B and -7B on AIME 2024.

Model	# Average Output Length			# Average Frequency of "Wait" and "wait"		
	TOTAL	CORRECT	INCORRECT	TOTAL	CORRECT	INCORRECT
DEEPSSEEK-R1-DISTILL-QWEN-1.5B	43176	21859	52629	109	49	138
DEEPSSEEK-R1-DISTILL-QWEN-7B	34193	19315	52376	75	35	124

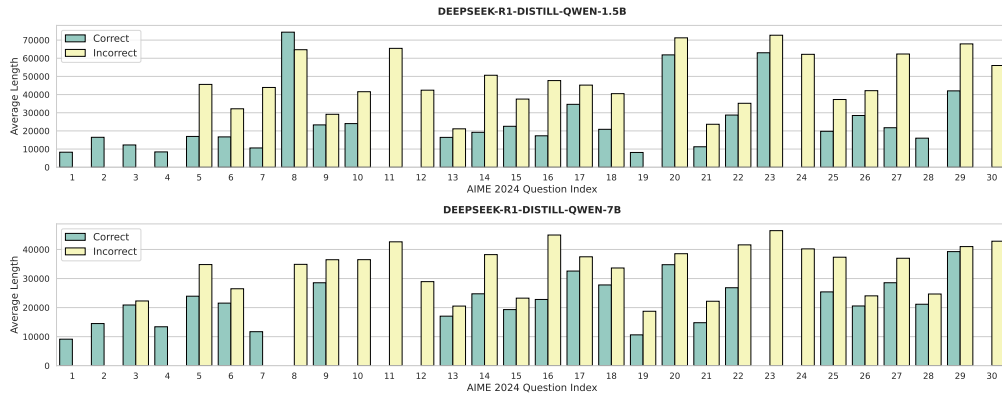


Figure 2: Distribution of average length for correct and incorrect answers under the same question.

3.2 Length-Aware Group Relative Policy Optimization (L-GRPO)

Recent studies [8, 11] have found that the reasoning response length is not strongly correlated with the correctness of the answer; that is, a long CoT reasoning response does not necessarily represent a

Table 2: Training Template. **{question}** will be replaced with the specific question during training.

A conversation between User and Assistant. The user asks a question, and the Assistant solves it. The assistant first thinks about the reasoning process in the mind and then provides the user with the answer. The reasoning process and answer are enclosed within <code><think></code> <code></think></code> and <code><answer></code> <code></answer></code> tags, respectively, i.e., <code><think></code> reasoning process here <code></think></code> <code><answer></code> answer here <code></answer></code>. Please reason step by step, and put your final answer within <code>\boxed{ }</code>. User: {question} Assistant:

correct result, and a short CoT reasoning response does not necessarily represent an incorrect one. On the contrary, the correct CoT reasoning responses are usually shorter in length, while incorrect reasoning responses tend to be longer. Therefore, we first analyze the response lengths of DEEPSEEK-R1-DISTILL-QWEN-1.5B and -7B, as shown in Table 1 and Figure 2. Table 1 shows that incorrect responses are obviously longer and include more detailed reasoning processes. Simultaneously, to differentiate whether the incorrect results all stemmed from complex problems, thereby causing the CoT for incorrect responses to be longer in the statistical results, we analyze the model’s responses at the sample level, as shown in Figure 2. Interestingly, in most cases, for the same question, correct responses are still shorter. Furthermore, we conduct a deeper analysis of the correct responses to the same question and find that correct answers may exhibit excessive reflection, which leads to long CoT, resulting in longer CoT responses, as shown in Figure 4.

Based on the above analysis, we reshape the reward function in GRPO. When the model’s rollout results for a question are all correct, we further optimize the model’s reasoning length for that question by using the remaining maximum response length as a reward (under the specified context length, the more remaining context length, the higher the reward), as calculated below.

$$\tilde{A}_i = \frac{\hat{r}_i - \text{mean}(\{\hat{r}_1, \hat{r}_2, \dots, \hat{r}_G\})}{\text{std}(\{\hat{r}_1, \hat{r}_2, \dots, \hat{r}_G\})}, \quad \hat{r}_i = r_i + \lambda \hat{\mathcal{L}}_i, \quad \hat{\mathcal{L}}_i = \begin{cases} 1 - \frac{\mathcal{L}_i}{\mathcal{L}_{\text{Max}}}, & \text{if } \sum_{i=1}^G r_i = G \\ 0, & \text{if } \sum_{i=1}^G r_i \neq G \end{cases} \quad (7)$$

where $\mathcal{L}_i$ is the length of the i -th response and $\mathcal{L}_{\text{Max}}$ indicates the max response length. Then, we optimize the policy model using the following objective:

$$\begin{aligned} \mathcal{J}_{\text{L-GRPO}}(\theta) &= \mathbb{E}_{q \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \\ &\frac{1}{G} \sum_{i=1}^G \left\{ \min \left[\tau_i(\theta) \hat{A}_i, \text{clip}(\tau_i(\theta), 1 - \varepsilon_l, 1 + \varepsilon_h) \hat{A}_i \right] - \beta \mathbb{D}_{\text{KL}}[\pi_{\theta} || \pi_{\text{ref}}] + \alpha \mathbb{H}(\pi_{\theta}) \right\}, \quad (8) \\ \text{s.t.} \quad &0 < |\{o_i | \text{is_equivalent}(o_i, a)\}|. \end{aligned}$$

3.3 Rule-based Reward Model

Using a trained reward model typically introduces the issue of reward hacking [24–29]. To mitigate this issue, we directly adopt the final accuracy from a verifiable task as the outcome reward, calculated according to the following rule:

$$r_i(o_i, a) = \begin{cases} 1, & \text{if is_equivalent}(o_i, a) \\ 0, & \text{if not is_equivalent}(o_i, a) \end{cases} \quad (9)$$

where a indicates the ground-truth answer and o_i contains the predicted answer. Additionally, it is important to note that the trained model must adhere strictly to the training prompt by generating the chain-of-thought within the `<think>``</think>` tags and subsequently presenting the final answer within the `<answer>``</answer>` tags with the boxed tag.

3.4 Training Dataset Curation

To select and curate high-quality data for scaling RL, we include challenging problems from DeepScaleR [8], DAPO-Math-17K [12], and MATH [16] to enhance problem difficulty and diversity in our data mixture:

Table 3: Overall performance on five competition-level reasoning benchmarks. Our models outperform prior state-of-the-art approaches with zero RL paradigm. [†] indicate the results from [20].

Model	AIME 2024	MATH-500	AMC 2023	Minerva	Olympiad	Avg. Score
Qwen2.5-1.5B-Base [21] [†]	0.0	3.3	2.5	1.8	1.5	1.82
Qwen2.5-1.5B-Instruct [21] [†]	1.3	57.5	26.2	19.4	20.3	24.9
Qwen2.5-Math-1.5B-Base [22] [†]	11.3	51.7	44.0	11.3	26.0	28.9
Qwen2.5-Math-1.5B-Instruct [22] [†]	12.0	74.7	26.7	35.0	37.9	37.3
DeepSeek-R1-Distill-Qwen-1.5B [5]	28.8	82.8	62.9	26.5	43.3	48.9
DeepScaleR-1.5B-Preview [8]	43.1	87.8	73.6	30.2	50.0	56.9
FastCuRL-1.5B-Preview [11]	43.1	88.0	74.2	31.6	50.4	57.5
FastCuRL-1.5B-V3 [11]	49.6	90.5	78.5	34.7	54.5	61.6
Qwen2.5-7B-Base [21] [†]	3.3	64.6	30.0	25.7	29.0	30.5
Qwen2.5-7B-Instruct [21] [†]	12.3	77.1	52.8	34.9	38.7	43.2
Qwen2.5-Math-7B-Base [22] [†]	20.7	64.3	56.2	17.3	29.0	37.5
Qwen2.5-Math-7B-Instruct [22] [†]	15.7	82.9	67.0	35.0	41.3	48.4
Eurus-2-7B-PRIME [23] [†]	17.8	80.1	63.0	37.5	43.9	48.5
Open-Reasoner-Zero-7B [24] [†]	19.7	83.9	59.5	31.6	47.6	48.5
SimpleRL-Zero-7B [25] [†]	14.0	77.9	58.0	33.0	39.0	44.4
SimpleRL-Zero-Math-7B [25] [†]	22.7	76.9	62.2	30.1	39.3	46.2
Oat-Zero-7B [13] [†]	28.0	79.4	66.2	34.4	43.8	50.4
ConciseR-Zero-7B-Preview (Stage-1)	42.8	83.0	73.9	31.8	45.1	55.3
ConciseR-Zero-7B (Stage-2)	43.3	83.0	76.7	31.5	46.0	56.1

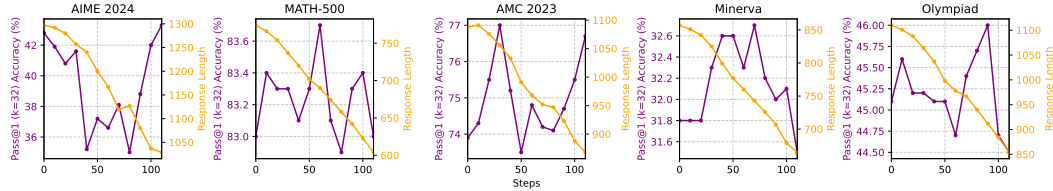


Figure 3: A detailed evaluation of accuracy and response length throughout the training steps.

DeepScaleR², which contains approximately 40K unique mathematics-specific problem-answer pairs collected from AIME (1984-2023), AMC (prior to 2023), Omni-MATH, and Still datasets [17, 18, 19].

DAPO-Math-17K³, which contains approximately 17K problem-answer pairs, each paired with an integer as the answer. DAPO-Math-17K was compiled from the Art of Problem Solving (AoPS⁴) website and official competition websites using a combination of web scraping and manual annotation.

MATH⁵ (Level 3-5), which contains approximately 8K problem-answer pairs. Each problem has a step-by-step solution which can be used to teach models to generate explanations.

After obtaining the above datasets, we employ Math-Verify⁶ to re-extract answers from the provided textual solutions, selecting only those cases where the extracted answer matches the corresponding answer in the dataset. We discard any samples that are empty, incomplete, or duplicates. Finally, we obtain approximately 59K reasoning problems as the training dataset. It should be noted that in the first stage, we use the 59K data to incentivize the model’s reasoning ability. Still, in the second stage, we use the MATH (Level 3-5) data as the training set to optimize the model’s reasoning length.

²<https://huggingface.co/datasets/agentica-org/DeepScaleR-Preview-Dataset>

³<https://huggingface.co/datasets/BytedTsinghua-SIA/DAPO-Math-17k>

⁴<https://artofproblemsolving.com/>

⁵https://huggingface.co/datasets/EleutherAI/hendrycks_math

⁶<https://github.com/huggingface/Math-Verify>

Question There exist real numbers x and y, both greater than 1, such that $\log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = 10$. Find xy. Response (a) (0.023 characters) To solve the problem, we start by analyzing the given equations: $\log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = 10$ $\log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 10$ We can use the properties of logarithms to simplify these equations. Recall that $\log_a(b^c) = c \log_a(b)$. Applying this property, we get: $\log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = 10 \implies \log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = 10$ $\log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 10 \implies \log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 10$ From the first equation, we have: $\log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = 10 \implies \log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = 10$ From the second equation, we have: $\log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 10 \implies \log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 10$ Next, we use the change of base formula for logarithms, which states that $\log_a(b) = \frac{\log_c(b)}{\log_c(a)}$. So, $\log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = \frac{\log(\log_{\lfloor \log(y) \rfloor})}{\log(\lfloor \log(x) \rfloor)}$. Substituting the expressions we found, we get: $\frac{\log(\log_{\lfloor \log(y) \rfloor})}{\log(\lfloor \log(x) \rfloor)} = 10 \implies \log(\log_{\lfloor \log(y) \rfloor}) = 10 \log(\lfloor \log(x) \rfloor)$ This simplifies to: $\log(\log_{\lfloor \log(y) \rfloor}) = 10 \log(\lfloor \log(x) \rfloor) \implies \log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$ Thus, the value of $\log_{\lfloor \log(y) \rfloor} 10$ is $\log_{\lfloor \log(x) \rfloor} 10$.	Response (b) (2,799 characters) To solve the problem, we start with the given equations: $\log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = 10 \quad \text{and} \quad \log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 10$ First, we simplify each logarithmic equation. Using the properties of logarithms, specifically that $\log_a(b^c) = c \log_a(b)$, we can rewrite the equations as: $\log_{\lfloor \log(x) \rfloor} (\log_{\lfloor \log(y) \rfloor}^2) = 10 \quad \text{and} \quad \log_{\lfloor \log(y) \rfloor} (\log_{\lfloor \log(x) \rfloor}^2) = 10$ From the first equation, we get: $\log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = 5 \implies \log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = 5$ From the second equation, we get: $\log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 5 \implies \log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 5$ We know that $\log_a(b) = \frac{\log_c(b)}{\log_c(a)}$. So substituting the expressions we have: $\log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = \frac{\log(\log_{\lfloor \log(y) \rfloor})}{\log(\lfloor \log(x) \rfloor)} = 5 \quad \text{and} \quad \log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = \frac{\log(\log_{\lfloor \log(x) \rfloor})}{\log(\lfloor \log(y) \rfloor)} = 5$ We get: $\log(\log_{\lfloor \log(y) \rfloor}) = 5 \log(\lfloor \log(x) \rfloor) \quad \text{and} \quad \log(\log_{\lfloor \log(x) \rfloor}) = 5 \log(\lfloor \log(y) \rfloor)$ This gives us the equation: $\log(\log_{\lfloor \log(y) \rfloor}) = 5 \log(\lfloor \log(x) \rfloor) \quad \text{and} \quad \log(\log_{\lfloor \log(x) \rfloor}) = 5 \log(\lfloor \log(y) \rfloor)$ Cross-multiplying to solve for $\log(x)$ in terms of $\log(y)$: $\log(\log_{\lfloor \log(y) \rfloor}) = 5 \log(\lfloor \log(x) \rfloor) \implies \log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$ So, we have found that $\log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$. To verify, we substitute $\log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$ back into the original logarithmic equations to ensure consistency. From $\log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$, we get $\log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$. Substituting $\log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$ into $\log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 5$, we get $\log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$. Substituting $\log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$ into $\log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = 5$, we get $\log_{\lfloor \log(x) \rfloor} 10 = 10 \log_{\lfloor \log(y) \rfloor} 10$. Thus, the value of $\log_{\lfloor \log(y) \rfloor} 10$ is $10 \log_{\lfloor \log(x) \rfloor} 10$.	Using the change of base formula $\log_a(b) = \frac{\log_c(b)}{\log_c(a)}$, we have: $\log_{\lfloor \log(x) \rfloor} \log_{\lfloor \log(y) \rfloor} = \frac{\log(\log_{\lfloor \log(y) \rfloor})}{\log(\lfloor \log(x) \rfloor)} = 10$ $\log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = \frac{\log(\log_{\lfloor \log(x) \rfloor})}{\log(\lfloor \log(y) \rfloor)} = 10$ we set it equal to $\frac{\log(\log_{\lfloor \log(y) \rfloor})}{\log(\lfloor \log(x) \rfloor)} = 10$ Cross-multiplying gives: $\log(\log_{\lfloor \log(y) \rfloor}) = 10 \log(\lfloor \log(x) \rfloor) \implies \log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$ Rearranging terms: $\log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10 \implies \log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$ This equation must hold true for some $\log(x) > 1$. Similarly, we check the second equation: $\log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 10 \implies \log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 10$ Substitute $\log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$ into the equation: $\log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = 10 \implies \log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$ Using the change of base formula again: $\log_{\lfloor \log(y) \rfloor} \log_{\lfloor \log(x) \rfloor} = \frac{\log(\log_{\lfloor \log(x) \rfloor})}{\log(\lfloor \log(y) \rfloor)} = 10$ Setting it equal to $\frac{\log(\log_{\lfloor \log(x) \rfloor})}{\log(\lfloor \log(y) \rfloor)} = 10$: $\log(\log_{\lfloor \log(x) \rfloor}) = 10 \log(\lfloor \log(y) \rfloor) \implies \log_{\lfloor \log(x) \rfloor} 10 = 10 \log_{\lfloor \log(y) \rfloor} 10$ Letting $\log_{\lfloor \log(y) \rfloor} 10 = z$, so $\log_{\lfloor \log(x) \rfloor} 10 = z$ and thus $\log_{\lfloor \log(x) \rfloor} 10 = z$: $\log_{\lfloor \log(x) \rfloor} 10 = z \implies \log_{\lfloor \log(x) \rfloor} 10 = z$ Since $\log_{\lfloor \log(y) \rfloor} 10 = z$, substituting $\log_{\lfloor \log(y) \rfloor} 10 = z$ into the equation: $\log_{\lfloor \log(x) \rfloor} 10 = z \implies \log_{\lfloor \log(x) \rfloor} 10 = z$ This equation needs to be consistent with the conditions given. Given the symmetry and the algebraic manipulations, it verifies that our solution $\log_{\lfloor \log(y) \rfloor} 10 = 10 \log_{\lfloor \log(x) \rfloor} 10$ is correct. Thus, the value of $\log_{\lfloor \log(y) \rfloor} 10$ is $10 \log_{\lfloor \log(x) \rfloor} 10$.
---	---	---

Figure 4: Comparison of responses of ConciseR-Zero-7B-Preview to the same question.

4 Experiments

Training Details. In this paper, we train our models using the verl⁷ framework [26] and leverage Qwen2.5-Math-7B [22] as the base model. During training, we utilize the Adam [39] optimizer with a constant learning rate of 1×10^{-6} . We leverage a batch size of 128 with each question generating 32 rollouts, the maximum response length is set to 3,072 tokens, and training is conducted using mini-batches of size 128. As for the Clip-Higher, similar to the prior work [12], we set the clipping parameter ε_l to 0.2 and ε_h to 0.28, which effectively balance the trade-off between exploration and exploitation for RL. Specifically, for GRPO++, we set the entropy coefficient α to 0.001. For L-GRPO, we set the KL penalty coefficient β to 0.01 and set the λ to 0.000002.

Evaluation Benchmarks. Similar to the prior work [13, 11], the performance of our models is evaluated on a diverse suite of competition-level benchmarks including AIME 2024⁸ (comprises 30 challenge problems), AMC 2023⁹ (contains 40 mathematical problems, covering algebra, geometry, number theory, and combinatorics), Minerva Math [17], MATH-500 [16] (is a challenging benchmark comprising competition-level problems), and OlympiadBench [27].

Evaluation Setup. In this paper, our two-stage RL training framework aims to enhance the reasoning performance while reducing the response length, thereby enabling more concise reasoning. To this end, we adopt the Pass@k evaluation metric, reporting Pass@1 accuracy computed with a non-zero sampling temperature. Therefore, we set the maximum response length to 3,072 tokens. Specifically, we select a temperature of 0.6 combined with a top-p value of 0.95 to generate multiple responses (typically 32 samples) for each query. The used training template is shown in Figure 2.

Baselines. We conduct comprehensive evaluations against several baselines with zero RL paradigm, including Qwen2.5 [21], Qwen2.5-Math [22], SimpleRL-Zero [25], Open-Reasoner-Zero-7B [24], Euror-2-7B-PRIME [23], and Oat-Zero-7B [13]. Furthermore, we also present the results of the models after RL based on the DeepSeek-R1-Distill-Qwen-1.5B model [5], including DeepScaleR-1.5B-Preview [8], FastCuRL-1.5B-Preview, and FastCuRL-1.5B-V3 [11].

4.1 Main Results

The experimental results reported in Table 3 clearly demonstrate that our proposed model, ConciseR, significantly outperforms existing baselines with zero RL paradigm on five widely recognized reasoning benchmarks. Specifically, ConciseR achieves an average accuracy improvement of 55.2% compared to the base model, Qwen2.5-Math-7B. Meanwhile, our method, GRPO++, also consistently surpasses all baselines, showing superior overall performance averaged across the five benchmarks.

Figure 3 illustrates changes in accuracy and response length during the training process of L-GRPO across the five benchmarks. As indicated, the average accuracy on each benchmark remains stable throughout training, without exhibiting any noticeable degradation. Interestingly, the average response

⁷<https://github.com/volcengine/verl>

⁸<https://huggingface.co/datasets/AI-MO/aimo-validation-aimc>

⁹<https://huggingface.co/datasets/AI-MO/aimo-validation-amc>

length on each benchmark consistently decreases, with reductions of 21%, 22%, 20%, 22%, and 23% observed on AIME 2024, MATH-500, AMC 2023, Minerva, and Olympiad benchmarks, respectively. This demonstrates that our training approach successfully maintains model accuracy while generating more concise and efficient responses.

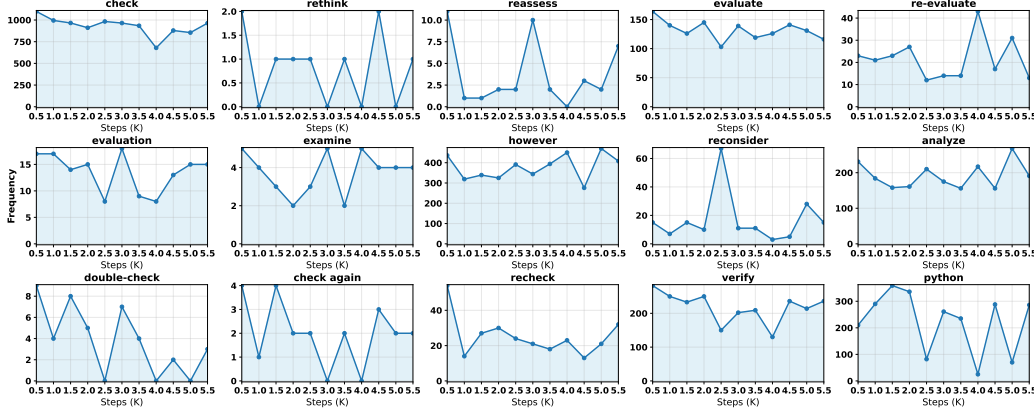


Figure 5: Count of keyword occurrences out of 14,022 responses (1558 questions $\times$ 11 test times).

5 Discussions

5.1 Analysis of Changes in Reasoning Patterns

Inspired by prior work that observes the model’s reflective behavior by constructing a keyword pool, we have built a carefully selected keyword pool to observe changes in the thinking patterns of the responses during training. In our experiment, the keyword pool is limited to: check, rethink, reassess, evaluate, re-evaluate, evaluation, examine, however, reconsider, analyze, double-check, check again, recheck, verify, and wait. Then, we present the occurrences of various keywords in the responses generated by different training stages and steps in Figure 5 and Figure 6. Interestingly, when comparing the first and second stages, the frequency with which the model uses code to verify results has significantly increased (as reflected in the frequency of the keyword "python"). The model may have discovered that verifying results by writing code is more efficient. Meanwhile, keywords like "re-check" have decreased relatively, and other keywords have remained unchanged.

5.2 Case Study

An interesting observation is that python code is used for verification during mathematical problem solving, e.g., Questions (a) and (b) in Figure 7. Specifically, for Question (a), the model utilizes program code to calculate the answer. For Question (b), the model first presents the solution process through mathematical reasoning and then spontaneously writes program code to verify the correctness of the approach. Such cases illustrate how models employ procedural reasoning to self-correct and engage in subsequent attempts.

5.3 Failure Experience

In this section, we discuss our failure experiences in reward shaping. These experiments could also be regarded as an ablation study in L-GRPO. During the initial design of L-GRPO, we consider directly comparing the generation length of samples within a group, assigning higher rewards to samples with relatively shorter CoT reasoning responses. We then combine the length score with the accuracy reward to encourage the trained model to obtain correct answers through shorter CoT reasoning responses, as illustrated by the following two equations,

$$\hat{r}_i = r_i + \lambda \hat{\mathcal{L}}_i, \quad \hat{\mathcal{L}}_i = \begin{cases} \frac{\text{Max}(\{\mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_G\}) - \mathcal{L}_i}{\text{Max}(\{\mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_G\}) - \text{Min}(\{\mathcal{L}_1, \mathcal{L}_2, \dots, \mathcal{L}_G\})}, & \text{if } \sum_{i=1}^G r_i = G \\ 0, & \text{if } \sum_{i=1}^G r_i \neq G \end{cases}, \quad (10)$$

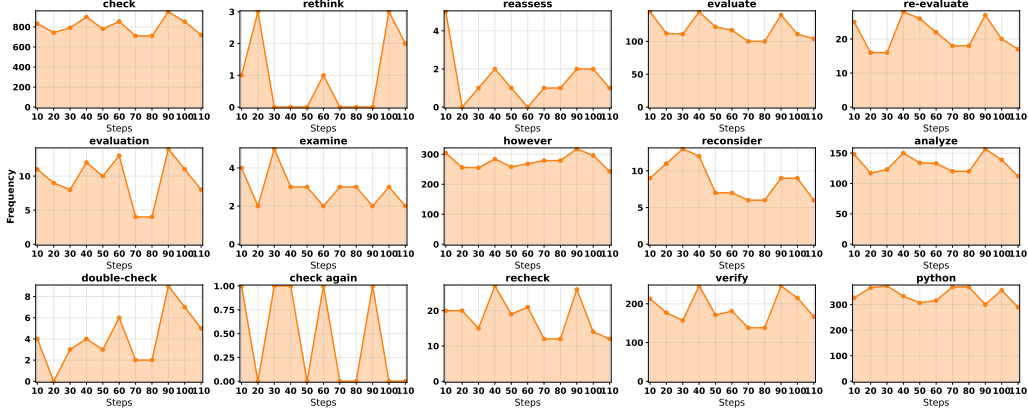


Figure 6: Count of keyword occurrences out of 15,580 responses (1558 questions $\times$ 11 test times).

Question (a) An integer, whose decimal representation reads the same left to right and right to left, is called symmetrical. For example, the number 513151315 is symmetrical, while 513152315 is not. How many nine-digit symmetrical numbers exist such that adding the number 11000 to them leaves them symmetrical? Response Let's write a Python code to iterate through all possible nine-digit symmetrical numbers and check if adding 11000 results in another symmetrical number. <pre> python count = 0 for a1 in range(1, 10): # a1 ranges from 1 to 9 for a2 in range(10): # a2 ranges from 0 to 9 for a3 in range(10): # a3 ranges from 0 to 9 for a4 in range(10): # a4 ranges from 0 to 9 for a0 in range(10): # a0 ranges from 0 to 9 N = int(f'{a1}{a2}{a3}{a4}{a3}{a2}{a1}{a0}{a0}') M = N + 11000 M_str = str(M) if M_str == M_str[::-1]: count += 1 print(count) ''' '''output 8100 ''' </pre>	Question (b) For positive integers a, b, $a \uparrow b$ is defined as follows: $a \uparrow 1 = a$, and $a \uparrow b = a \uparrow (a \uparrow (b-1))$ if $b > 1$. Find the smallest positive integer n for which there exists a positive integer a such that $a \uparrow n \equiv a \uparrow 6 \pmod n$. Response <pre> ### Step-by-Step Check ... #### Continue this process programmatically for efficiency: We write a Python script to automate the checking for the smallest $(n \setminus)$: python import sympy as sp def tetration_mod(a, b, n): if b == 1: return a % n else: exp = tetration_mod(a, b-1, n) return pow(a, exp, n) def find_smallest_n(): for n in range(2, 100): for a in range(1, n): if tetration_mod(a, 6, n) != tetration_mod(a, 7, n): return n return None smallest_n = find_smallest_n() print(smallest_n) '''output 2 ''' </pre>
Question (c) Given that the sequence starts with 2 and alternates by adding 2 between consecutive terms, find the 30th term of this arithmetic sequence. Response <pre> think>a30 = 2 + 29*2 = 60</think> answer>\boxed{60}</answer> </pre>	

Figure 7: Illustration of cases.

$$\hat{r}_i = \frac{G_{r_i \neq 0} - 1}{G_{r_i \neq 0}} r_i + \frac{\hat{\mathcal{L}}_i}{\sum_{i=1}^G \hat{\mathcal{L}}_i}, \quad \hat{\mathcal{L}}_i = \begin{cases} 0, & \text{if is_equivalent}(o_i, a) \\ 1 - \frac{\mathcal{L}_i}{\sum_{i=1}^G \mathcal{L}_i}, & \text{if not is_equivalent}(o_i, a) \end{cases} \quad (11)$$

However, we find that this direct rewarding easily causes the model to skip the reasoning process and immediately start guessing answers, manifesting as an empty reasoning response within the `<think></think>` tags while directly outputting the final answer within the `<answer></answer>` tags. On the contrary, indirectly using the maximum context length to design the reward function can, to some extent, avoid the issues mentioned above.

6 Related Work

Recent advances in reinforcement learning have significantly enhanced the reasoning capabilities of large language models. A pivotal development in this domain is OpenAI's o1 [4], which leverages large-scale RL training to promote CoT reasoning. This approach has resulted in notable improvements in complex mathematics and coding benchmarks. DeepSeek-R1 [5] demonstrates that pure RL

post-training via GRPO, without the need for supervised warm-up, can directly induce robust reasoning abilities. Remarkably, this kind of method not only achieves performance competitive with o1 but also exhibits emergent behaviors such as self-verification and multi-step planning. This paradigm shift significantly reduces memory and computational overhead compared to earlier GRPO implementations [24, 25, 28], all while maintaining competitive performance levels. Recent algorithmic variants have focused on enhancing training efficiency [8, 9, 11, 12, 13, 14, 25, 29], yet they preserve GRPO’s core methodology of parallel CoT sampling across groups. These advancements collectively contribute to more efficient and robust training methodologies for LLMs, thereby enhancing their reasoning capabilities and performance on complex tasks.

7 Conclusion

In this paper, we propose ConciseR, which introduces a simple yet effective two-stage reinforcement learning framework. First, it incentivizes the model’s reasoning capabilities via GRPO++, and then it reduces the model’s response length to improve the quality of the CoT response implicitly via L-GRPO. *Importantly, we innovatively propose that during training, response length optimization is only triggered when all rollouts for a given training sample are correct. This embodies the "walk before you run" principle.* Experiments demonstrate that ConciseR consistently achieves the best efficiency-accuracy synergistic improvement, significantly outperforming existing efficient reasoning methods across five benchmarks.

References

- [1] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024.
- [2] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori Hashimoto. s1: Simple test-time scaling. *arXiv preprint arXiv:2501.19393*, 2025.
- [3] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed H. Chi, Quoc V Le, and Denny Zhou. Chain of thought prompting elicits reasoning in large language models. In Alice H. Oh, Alekh Agarwal, Danielle Belgrave, and Kyunghyun Cho, editors, *Advances in Neural Information Processing Systems*, 2022.
- [4] Aaron Jaech, Adam Kalai, Adam Lerer, Adam Richardson, Ahmed El-Kishky, Aiden Low, Alec Helyar, Aleksander Madry, Alex Beutel, Alex Carney, et al. Openai o1 system card. *arXiv preprint arXiv:2412.16720*, 2024.
- [5] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [6] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.
- [7] Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. Demystifying long chain-of-thought reasoning in llms, 2025.
- [8] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Tianjun Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://github.com/agentica-project/deepscaler>, 2025. Notion Blog.
- [9] Kimi Team, Angang Du, Bofei Gao, Bowei Xing, Changjiu Jiang, Cheng Chen, Cheng Li, Chenjun Xiao, Chenzhuang Du, Chonghua Liao, et al. Kimi k1. 5: Scaling reinforcement learning with llms. *arXiv preprint arXiv:2501.12599*, 2025.
- [10] Alejandro Cuadron, Dacheng Li, Wenjie Ma, Xingyao Wang, Yichuan Wang, Siyuan Zhuang, Shu Liu, Luis Gaspar Schroeder, Tian Xia, Huanzhi Mao, Nicholas Thumiger, Aditya Desai, Ion Stoica, Ana Klimovic, Graham Neubig, and Joseph E. Gonzalez. The danger of overthinking: Examining the reasoning-action dilemma in agentic tasks, 2025.
- [11] Mingyang Song, Mao Zheng, Zheng Li, Wenjie Yang, Xuan Luo, Yue Pan, and Feng Zhang. Fastcurl: Curriculum reinforcement learning with progressive context extension for efficient training rl-like reasoning models, 2025.
- [12] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025.
- [13] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- [14] Mehdi Fatemi, Banafsheh Rafiee, Mingjie Tang, and Kartik Talamadupula. Concise reasoning via reinforcement learning. *arXiv preprint arXiv:2504.05185*, 2025.
- [15] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [16] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

- [17] Aitor Lewkowycz, Anders Andreassen, David Dohan, Ethan Dyer, Henryk Michalewski, Vinay Ramasesh, Ambrose Slone, Cem Anil, Imanol Schlag, Theo Gutman-Solo, et al. Solving quantitative reasoning problems with language models. *Advances in Neural Information Processing Systems*, 35:3843–3857, 2022.
- [18] Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. Omni-math: A universal olympiad level mathematic benchmark for large language models. *CoRR*, abs/2410.07985, 2024.
- [19] Yingqian Min, Zhipeng Chen, Jinhao Jiang, Jie Chen, Jia Deng, Yiwen Hu, Yiru Tang, Jiapeng Wang, Xiaoxue Cheng, Huatong Song, Wayne Xin Zhao, Zheng Liu, Zhongyuan Wang, and Ji-Rong Wen. Imitate, explore, and self-improve: A reproduction report on slow-thinking reasoning systems, 2024.
- [20] Andreas Hochlehnert, Hardik Bhatnagar, Vishaal Udandaraao, Samuel Albanie, Ameya Prabhu, and Matthias Bethge. A sober look at progress in language model reasoning: Pitfalls and paths to reproducibility, 2025.
- [21] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [22] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. Qwen2.5-math technical report: Toward mathematical expert model via self-improvement, 2024.
- [23] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, Qixin Xu, Weize Chen, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.
- [24] Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-reasoner-zero: An open source approach to scaling up reinforcement learning on the base model, 2025.
- [25] Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild, 2025.
- [26] Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, page 1279–1297. ACM, March 2025.
- [27] Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, et al. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3828–3850, 2024.
- [28] Hugging Face. Open r1: A fully open reproduction of deepseek-r1, January 2025.
- [29] Liang Wen, Yunke Cai, Fenrui Xiao, Xin He, Qi An, Zhenyu Duan, Yimin Du, Junchen Liu, Lifu Tang, Xiaowei Lv, et al. Light-r1: Curriculum sft, dpo and rl for long cot from scratch and beyond. *arXiv preprint arXiv:2503.10460*, 2025.